

Claude Worker Pipeline — Spec

Sep 25, 2026 · @Master

Goal

A Vikunja task marked ready becomes a reviewed pull request without anyone at a terminal. One Claude Code worker per task, N tasks in parallel on one Mac, each in its own git worktree with its own ports and database namespace.

What it does:

- Picks up tasks from Vikunja, runs a Claude Code session per task, and writes status back.
- Refuses to open a PR until the branch's tests pass and a second review pass has run.
- Notifies on "needs input" and "done"; caps spend per task and per day.

What it deliberately does not do:

- Merge. A human merges every PR.
- Coordinate workers. Tasks are assumed independent; conflicts surface at merge, not during work.
- Run in the cloud. Everything executes on the developer's Mac.

Architecture

Six components, all local. The dispatcher is the only new code; the rest is Claude Code, ntree, gh and hooks wired together.

```

flowchart LR
    V[Vikunja task\nstatus=ready] --> D[Dispatcher\nPython daemon]
    D --> W[ntree new slug\nworktree + ports + deps]
    W --> C[claude -p\nunattended session]
    C --> G[Verification gate\ntests + review]
    G -- green --> P[gh pr create]
    G -- red --> C
    P --> H[Human merge]
    H --> X[Cleanup\nntree rm]
    D -. status, cost .-> V
    C -. stream-json .-> D
  
```

Task flow:

1. Dispatcher sees a Vikunja task in the `ready` bucket and claims it (moves to `in-progress`, records its own PID).
2. `ntree new <slug> --from main` creates the worktree, reserves ports, runs `NTREE_SETUP_CMD`.
3. Dispatcher launches `claude -p` inside the worktree with the task prompt, a turn cap and `--output-format stream-json`.
4. A Stop hook runs the test command via `ntree run <slug> -- <tests>`; failure returns the session to work.
5. On green, a second `claude -p` reviews the diff read-only and posts findings; blocking findings send it back once.
6. `gh pr create` from the worktree, body linking the Vikunja task and session ID. Task moves to `review`.
7. On PR merge or close, dispatcher runs `ntree rm <slug>` and moves the task to `done` or `blocked`.

Components:

Component	What it is	Owns
Dispatcher	Python daemon, ~200 lines, systemd/launchd	Vikunja polling, prompt build, session launch, budget, notifications
Worker	<code>claude -p</code> with settings.json + hooks	The change, commits, PR body draft
Workspace	ntree worktree	Files, ports, <code>.venv</code> , DB namespace
Gate	Stop hook + review session	Test result, review verdict
PR step	<code>gh</code> CLI	Branch push, PR creation
Cleanup	Dispatcher on merge + nightly <code>ntree doctor</code>	Worktrees, ports, orphan DBs

Dispatcher

A single Python process that turns Vikunja tasks into Claude Code sessions and keeps Vikunja truthful about them. Poll first; add a webhook later if latency matters.

Intake. Poll the Vikunja API every 60 s for tasks with label `claude` and bucket `ready` . Claim by moving to `in-progress` and writing a comment `dispatched: session=<id> pid=<pid> workspace=<slug>` . A task with an existing `dispatched: comment` is never re-claimed.

Prompt construction. Title + description + checklist items, plus a fixed preamble:

- Branch name and workspace slug (from the task ID: `task-123-short-title`).
- "Commit as you go. Do not push. Do not open a PR — the pipeline does that."
- "If blocked, write BLOCKED: <reason> as your final message and stop."
- Acceptance criteria copied verbatim from the task checklist.

Launch. From inside the worktree:

```
claude -p "$PROMPT" \
  --output-format stream-json \
  --max-turns 60 \
  --permission-mode auto \
  --settings .claude/worker-settings.json
```

Concurrency: `MAX_WORKERS=4` by default; a fifth ready task waits. Each worker is a subprocess the dispatcher owns, so a dispatcher restart reaps orphans on startup by PID.

Status write-back. Every state change is a Vikunja comment and a bucket move:

Event	Bucket	Comment
Claimed	in-progress	dispatched: ...
Tests failed (retry n)	in-progress	gate: tests red, retry n/2
Review blocked	in-progress	gate: review blocking, retry
PR opened	review	pr: <url>
Worker wrote BLOCKED	blocked	blocked: <reason>
Turn/budget cap hit	blocked	cap: turns=60 or cost=\$X
PR merged	done	merged: <sha>
PR closed unmerged	blocked	closed without merge

Budget caps. Three, all enforced by the dispatcher, not by the model:

- Per task: `--max-turns 60` and a wall-clock limit of 45 min. Either hit moves the task to `blocked`.
- Per task retries: at most 2 test-red retries and 1 review retry, then `blocked`.
- Per day: sum the `cost_usd` field from each session's final stream-json `result` event; stop dispatching at `DAILY_CAP_USD` and post once to Slack.

State. One SQLite file: task ID, session ID, workspace, PID, started_at, cost, state. Vikunja is the source of truth for the queue; SQLite is only for restart recovery and the cost ledger.

Worker isolation

Each task gets a real checkout, its own ports and its own database name, so four workers can run tests at once without colliding. `ntree` provides the first two; the app has to honour the third.

Worktree. `ntree new <slug> --from main` under `.ntree/<slug>/`. Shared `.git` object store, so history costs nothing per workspace. The dispatcher, not Claude, creates and removes worktrees; Claude is launched with the worktree as its working directory and never told about `.ntree/`.

Ports. `.ntreerc` per repo:

```
NTREE_SETUP_CMD='uv sync'
NTREE_PORT_RANGE_START=8100
NTREE_PORT_RANGE_END=8199
NTREE_PORTS='API FRONTEND'
```

`ntree` writes `.env.workspace` with `PORT`, `PORT2`, `API_PORT`, `FRONTEND_PORT` and `NTREE_WORKSPACE`. Every command the gate runs goes through `ntree run <slug> -- ...` so the env is sourced.

Database. The repo's settings must read `NTREE_WORKSPACE` and derive a DB name from it. For Django: `NAME = f"app_{os.environ.get('NTREE_WORKSPACE', 'dev')}"`. The setup command creates it (`createdb` or `manage.py migrate` against a Postgres that allows it). Redis, Celery queues and any other shared service get the same prefix. If the app cannot be namespaced this way, the repo is not ready for parallel workers; fix that first.

Dependencies. `uv sync` (or `npm ci`) per workspace via `NTREE_SETUP_CMD`. No COW cloning of `.venv`; the README is right that it breaks on absolute paths. `ntree sync-deps` after any lockfile change on `main`.

What is shared. Git objects, the Postgres server process, the Anthropic auth token, `~/.claude/` settings. Nothing else. A worker that needs a fixture or secret gets it from `.env.workspace` or the repo, never from another workspace.

Portability note. ntree is macOS-only. On a Linux runner (see Rollout, phase 3) the same contract — worktree, port pair, `NTREE_WORKSPACE` — is provided by a 50-line shell equivalent that skips the APFS path. Keep the dispatcher talking to that contract, not to ntree's CLI directly.

Unattended permissions

Auto mode inside the sandbox, with a short deny list. A session that hits a permission prompt with nobody there is a stalled task, so every prompt path has to be closed before the first real run.

Mode. `--permission-mode auto`. The server-side classifier approves ordinary edits and commands and blocks destructive ones; anything it cannot classify falls back to a prompt, which in `-p` mode is a fail. `dontAsk` with a hand-written allow list is the fallback if auto mode is unavailable on the account (Bedrock/gateway routing) — it is stricter and needs more tuning, but it never hangs.

Sandbox. Filesystem writes limited to the worktree; network limited to the package registries, the Anthropic API and the local Postgres port. `.claude/worker-settings.json` in the repo:

```
{
  "permissions": {
    "defaultMode": "auto",
    "deny": [
      "Bash(git push*)",
      "Bash(gh pr*)",
      "Bash(rm -rf*)",
      "Bash(ntree*)",
      "Edit(.ntreerc)",
      "Edit(.claude/**)",
      "Edit(.github/**)"
    ]
  },
  "sandbox": {
    "enabled": true,
    "failIfUnavailable": true,
```

```

    "network": {
      "allowedDomains": ["api.anthropic.com", "pypi.org",
        "files.pythonhosted.org", "registry.npmjs.org"],
      "allowLocalBinding": true
    }
  }
}

```

The deny list is the contract: the worker cannot push, cannot open PRs, cannot touch the pipeline's own config, cannot run ntree. Those belong to the dispatcher and the gate.

Protected paths. `.env*`, `secrets/`, anything under `infra/` are read-deny as well as write-deny. If a task needs a config change there, it is not a task for this pipeline.

Approvals stated in the prompt. Auto mode honours boundaries written in conversation, so the preamble also says: "Do not modify migrations that already exist. Do not delete tests." These are cheaper than deny rules and easier to change per task.

Failure behaviour. A blocked action is logged to stream-json as a permission denial. The dispatcher counts them; three in one session moves the task to `blocked` with the denied command in the comment, because a worker fighting the deny list is a worker that misunderstood the task.

Verification gate

Two checks stand between "the worker says it is done" and a PR: the branch's own tests, then a review by a session that did not write the code. Both are mechanical; neither trusts the worker's summary.

Tests as a Stop hook. In `.claude/worker-settings.json`:

```
{
  "hooks": {
    "Stop": [{
      "hooks": [{
        "type": "command",
        "command": "ntree run $NTREE_WORKSPACE -- uv run pytest -x -q 2>&1 |
tail -40; exit ${PIPESTATUS[0]}",
        "timeout": 600
      }]
    }]
  }
}
```

A non-zero exit blocks the stop and feeds the tail of the output back as the next turn, so the worker sees the failure and keeps going. The Stop hook has a block cap; the dispatcher's retry limit (2) is lower than it, so the dispatcher decides, not the hook. Lint and type-check (`ruff` , `mypy`) run in the same command when the repo has them.

Working-tree check. Before review, the dispatcher runs `git status --porcelain` in the worktree. Uncommitted changes mean the worker stopped mid-way; it gets one more turn with "commit or discard your remaining changes".

Adversarial review. A second `claude -p` in the same worktree, read-only (`--permission-mode plan` or a deny on Edit/Write), with:

- The task description and acceptance criteria.
- `git diff main...HEAD`.
- The instruction: "Find reasons this should not merge. Output JSON: {verdict: pass|block, findings: [{severity, file, line, note}]}". Block only on correctness, security or a missed acceptance criterion; style is a note, not a block."

`--json-schema` enforces the shape. A `block` verdict sends the findings to the worker as a new prompt for one retry; a second `block` moves the task to `blocked` with the findings in the Vikunja comment. `pass` findings go into the PR body under "Review notes".

What the gate does not do. It does not run the app or click through it. If a task needs manual QA, the task says so and the PR gets a `needs-qa` label; the gate is not a substitute.

PR creation and merge policy

The dispatcher pushes and opens the PR; the worker never does. A human merges.

Push and create. From the worktree, after a green gate:

```
git push -u origin task-123-short-title
gh pr create --base main --title "<task title>" --body-file pr-body.md --
label claude-worker
```

`pr-body.md` is assembled by the dispatcher, not written by the worker:

- Vikunja task link and ID.
- Acceptance criteria as a checklist, each ticked by the review session's verdict.
- Worker's own summary (its final message, capped at 300 words).
- Review notes from the gate.
- Session ID and cost, for tracing.

Merge policy. Human merges, always. CI on the PR is the third check after the local gate and review. Squash-merge so the worker's commit trail does not land on `main`.

Follow-ups on the PR. Review comments on the PR are not handled by this pipeline in phase 1; the reviewer edits the branch or writes a new Vikunja task that references the PR. Auto-fix from PR comments is a phase 2 item once cloud sessions or a PR-triggered routine are in play.

Cleanup

Everything a task creates is removed when its PR closes, and a nightly pass catches what the event-driven path missed.

On PR merge or close. Dispatcher polls open `claude-worker` PRs every 5 min. When one closes:

1. `ntree rm <slug> --force` — stops any process, frees ports, removes the worktree.
2. `dropdb app_<slug>` and flush the matching Redis prefix.
3. `git branch -D <branch>` locally; the remote branch is deleted by GitHub's auto-delete setting.
4. Vikunja task to `done` (merged) or `blocked` (closed unmerged), with the outcome as a comment.

Nightly. A launchd job at 03:00:

- `ntree doctor` — prunes vanished worktrees, reconciles PIDs, clears stale locks.

- List workspaces older than 7 days with no open PR; remove them and post the list to Slack.
- Drop databases matching `app_task-*` with no live workspace.
- Rotate `.ntree/*/ntree.log` and the dispatcher's own log.

Disk budget. With `uv` hardlinking from its cache, a Django workspace costs roughly the checkout plus build output. Alert when `.ntree/` passes 10 GB.

Project context (CLAUDE.md)

Most "the agent did something dumb" failures are missing context, not model error. A repo is not eligible for the pipeline until its `CLAUDE.md` answers these, in under 150 lines:

Section	Must say
Layout	Where apps, services, models, tests and migrations live; what is generated and must not be edited
Run tests	The exact command the Stop hook uses, and how to run one file
Conventions	Formatter, linter, type-checker, commit message shape, branch naming
Isolation	That <code>NTREE_WORKSPACE</code> and <code>\$PORT</code> exist and must be used for anything that binds or names
Do not touch	Existing migrations, <code>infra/</code> , <code>.env*</code> , vendored code, the pipeline's own <code>.claude/</code> files
Definition of done	Tests pass, no new lint errors, acceptance criteria met, changes committed, nothing pushed
Blocked protocol	Write <code>BLOCKED: <reason></code> and stop; never guess at a missing requirement

Per-directory `CLAUDE.md` files for parts of a repo with their own rules (Kippr's Django API vs its SwiftUI client, say). Keep task-specific detail out of `CLAUDE.md` and in the task itself; the file describes the repo, the task describes the change.

Observability and notifications

The dispatcher tails every session and turns events into Vikunja comments and Slack messages, so the developer never has to open a terminal to know where a task stands.

Stream. `--output-format stream-json` gives one JSON line per event. The dispatcher keeps the raw stream at `.ntree/<slug>/session.jsonl` and extracts:

- `assistant` text turns → a rolling "last message" on the task.
- `tool_use` counts by tool → a cheap progress signal.
- permission denials → the counter from the Permissions section.
- the final `result` event → `cost_usd`, `duration_ms`, `num_turns`, and the summary for the PR body.

Vikunja. One comment per state change (table in Dispatcher). No per-turn comments; that is noise.

Slack. A single channel, `#claude-workers`, with a message on:

- task blocked (with reason and the Vikunja link)
- PR opened (with the PR link)
- daily cap reached
- nightly cleanup summary

Nothing on start or on test retries. Push notifications come for free if the channel is set to notify.

Dashboard. Deferred. `sqlite3 pipeline.db 'select ...'` and `ntree list` cover phase 1. If a dashboard is wanted later it reads the SQLite file; nothing else needs to change.

Transcripts. Full session transcripts stay in `~/.claude/projects/` per Claude Code's normal storage and are the audit trail. Retention follows the machine's policy, not the pipeline's.

Rollout

Three phases, each gated on a week of real use. Phase 1 is the whole value; phases 2 and 3 are only worth doing if phase 1 holds up.

Phase 1 — one repo, on the laptop. Target: finops (Python, FastAPI, small test suite, no shared services). Build in this order:

1. `.ntreerc`, `NTREE_WORKSPACE`-aware DB naming, `CLAUDE.md` to the table above.

2. `worker-settings.json` : auto mode, sandbox, deny list. Run one task by hand with `claude -p` and confirm zero prompts.
3. Stop hook with tests. Confirm a deliberately failing test blocks the stop.
4. Dispatcher: poll, claim, launch, write-back, caps. SQLite ledger.
5. Review session and PR creation.
6. Cleanup on close, nightly job, Slack.

Exit criteria: 10 tasks dispatched, at least 7 merged with no manual fix-up, no task stalled on a prompt, daily cost under the cap.

Phase 2 — second repo, harder isolation. Target: Kippr's Django backend (Postgres, migrations, a Swift client the worker cannot test). Adds: per-workspace DB creation in the setup command, `needs-qa` labelling, PR-comment follow-up as new tasks. Exit: same numbers as phase 1.

Phase 3 — every personal repo, plus PR follow-ups. Once phases 1 and 2 hold: enable the pipeline on ClawdBot Link and the remaining side projects, and add PR-comment follow-up — a review comment on a `claude-worker` PR becomes a new Vikunja task pointing at the branch, so the same worktree is resumed instead of a fresh one. Consider auto-merge for finops only if phase 1 and 2 numbers stay boring.

Open questions and out of scope

Decided:

- Scope is personal projects only; no gateway, auto mode is available. RBC and Gravity are out.
- Vikunja bucket moves are the state signal; labels only mark eligibility (`claude`).
- Postgres per workspace via `createdb` in the setup command. Template cloning only if setup time becomes the bottleneck.
- Review session runs a cheaper model than the worker, with the stricter JSON-schema prompt.
- No ntree fork. Linux portability is not needed while everything runs on the Mac.

Out of scope for all three phases:

- Agent teams and cross-session messaging. Tasks are independent by design; if two need coordination, they are one task.

- Cloud sessions and Claude Projects. Same goal, different data boundary; revisit if the self-hosted path stalls.
- Auto-merge, until phase 3, and then only for finops.